and create extension methods for them, as in listing 9.6. This approach creates an equivalent to the framework's `AddRazorPages()` extension method—a nice, simple registration API. As you add more features to your app, I think you'll appreciate it too.

Listing 9.6 Creating an extension method to tidy up adding multiple services

```
public static class EmailSenderServiceCollectionExtensions
{
    public static IServiceCollection AddEmailSender(
        this IServiceCollection services)                    ❶
    {
        services.AddScoped<IEmailSender, EmailSender>();     ❷
        services.AddSingleton<NetworkClient>();              ❷
        services.AddScoped<MessageFactory>();                ❷
        services.AddSingleton(                               ❷
            new EmailServerSettings                          ❷
            (                                                ❷
                host: "smtp.server.com",                     ❷
                port: 25                                     ❷
            ));                                              ❷
        return services;                                     ❸
    }
}
```

❶ Creates an extension method on IServiceCollection by using the "this" keyword

❷ Cuts and pastes your registration code from Program.cs

❸ By convention, returns the IServiceCollection to allow method chaining

With the preceding extension method created, the following listing shows that your startup code is much easier to grok!

Listing 9.7 Using an extension method to register your services

```
WebApplicationBuilder builder = WebApplication.CreateBuilder(args);

builder.Services.AddEmailSender();     ❶
```